ESCUELA DE INGENIERÍA INFORMÁTICA



PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

# METAHEURÍSTICA SIMULATED ANNEALING PARA JOB SHOP PROBLEM


AGUSTÍN GUZMÁN

NICOLÁS LEIVA

CRISTOBAL PINTO


INVESTIGACIÓN DE OPTIMIZACIÓN

PROFESORA: LESLIE PÉREZ CÁCERES

NOVIEMBRE 2026

# RESUMEN

El presente informe describe la implementación y evaluación de la metaheurística *Simulated Annealing* para resolver el problema de programación de talleres de trabajo (*Job Shop Problem*, JSP). El objetivo es minimizar el tiempo total de término de todos los trabajos, conocido como *makespan*, respetando el orden tecnológico de las operaciones de cada trabajo y la capacidad de cada máquina. La solución se representa mediante una secuencia de identificadores de trabajo que siempre es factible y se decodifica con una regla de programación que asigna cada operación al primer instante disponible de su máquina y de su trabajo. El vecindario se genera intercambiando dos posiciones de la secuencia y la aceptación de soluciones peores sigue el criterio de Metropolis, con un esquema de enfriamiento geométrico.

La implementación se realizó en Rust y se evaluó sobre tres instancias de complejidad creciente: $6 \times 6$, $15 \times 10$ y $50 \times 10$. Para cada instancia se ejecutó el algoritmo diez veces con los mismos parámetros. La mejor solución encontrada coincidió con el valor de referencia en las tres instancias: 55, 1032 y 2794, respectivamente. La instancia de tamaño medio presentó la mayor variabilidad, con una media de 1038,90 y una desviación estándar de 12,07, mientras que las instancias fácil y difícil alcanzaron ese valor en las diez ejecuciones. Los resultados muestran que la metaheurística es simple, robusta y computacionalmente económica para las instancias estudiadas.

**Palabras clave:** Job Shop Problem, Simulated Annealing, metaheurística, makespan, optimización combinatoria.

# ÍNDICE GENERAL

**Resumen/Abstract** **i**

**Índice General** **ii**

**Lista de Figuras** **iv**

**Lista de Tablas** **v**

**Lista de Algoritmos** **vi**

**Lista de Símbolos** **vii**

**1 Descripción general** **1**

1.1 Job Shop Problem . . . . . 1

1.2 Metaheurística seleccionada . . . . . 1

1.2.1 Tipo de metaheurística . . . . . 1

1.2.2 Ventajas y desventajas . . . . . 2

1.2.3 Razón de la elección . . . . . 2

**2 Implementación** **4**

2.1 Arquitectura del software . . . . . 4

2.2 Representación de soluciones . . . . . 5

2.3 Decodificación y evaluación . . . . . 5

2.4 Generación de la solución inicial . . . . . 6

2.5 Vecindario . . . . . 6

2.6 Criterio de aceptación . . . . . 6

2.7 Esquema de enfriamiento . . . . . 6

2.8 Criterios de parada . . . . . 7

2.9 Pseudocódigo . . . . . 7

2.10 Diagrama de flujo . . . . . 7

2.11 Parámetros y presupuesto computacional . . . . . 8

2.12 Reproducibilidad . . . . . 10

**3 Mejor solución** **11**

**4 Resultados** **12**

4.1 Configuración experimental . . . . . 12

4.2 Resultados por ejecución . . . . . 12

4.3 Resumen estadístico . . . . . 12

4.4 Análisis de resultados . . . . . 13

4.5 Gráfico de cajas . . . . . 13

**5 Conclusiones** **15**

5.1 ¿Qué aprendió implementando la metaheurística? . . . . . 15

5.2 ¿Cuál fue el desafío más grande de implementar la metaheurística? . . . . . 15

5.3 ¿Tiene alguna idea para mejorar los resultados? . . . . . 15

5.4 ¿Cree que este tipo de técnicas puede ser aplicada en la realidad chilena? . . . . . 16

5.5 Conclusión general . . . . . 16

**Referencias** **17**

# LISTA DE FIGURAS

2.1 Diagrama de flujo del *Simulated Annealing* implementado. . . . . 8

4.1 Gráfico de cajas del makespan obtenido en diez ejecuciones por instancia. . . . . 14

# LISTA DE TABLAS

2.1 Parámetros del *Simulated Annealing* utilizado en los experimentos. . . . . 9

3.1 Mejor solución encontrada para cada instancia. . . . . 11

4.1 Makespan obtenido en diez ejecuciones independientes por instancia. . . . . 12

4.2 Resumen estadístico de las diez ejecuciones por instancia. . . . . 13

# LISTA DE ALGORITMOS

2.1 *Simulated Annealing* para el JSP . . . . . 7

# LISTA DE SÍMBOLOS

| | |
|---|---|
| $\alpha$ | Factor de enfriamiento geométrico |
| $\Delta$ | Variación del makespan al pasar de una solución a un vecino |
| $C_i$ | Instante de término del trabajo $i$ |
| $C_{\max}$ | Makespan o instante de término de todos los trabajos |
| $J_i$ | Trabajo $i$, con $i = 1, \ldots, n$ |
| $m$ | Número de máquinas de la instancia |
| $m_{ij}$ | Máquina requerida por la operación $O_{ij}$ |
| $n$ | Número de trabajos de la instancia |
| $O_{ij}$ | Operación $j$-ésima del trabajo $i$ |
| $p_{ij}$ | Duración de la operación $O_{ij}$ |
| $s_{ij}$ | Instante de inicio de la operación $O_{ij}$ |
| $T$ | Temperatura actual del *Simulated Annealing* |
| $T_0$ | Temperatura inicial |
| $T_{\min}$ | Temperatura mínima de término |
| $x$ | Secuencia de trabajos que representa una solución |
| $x^*$ | Mejor secuencia encontrada durante la ejecución |

# 1. DESCRIPCIÓN GENERAL

## 1.1. Job Shop Problem

Formalmente, una instancia del JSP está compuesta por un conjunto de $n$ trabajos y un conjunto de $m$ máquinas. Cada trabajo $J_i$, con $i = 1, \ldots, n$ debe ejecutar una secuencia ordenada de operaciones

$$O_{i1} \to O_{i2} \to \cdots \to O_{im_i},$$

donde $m_i$ es el número de operaciones del trabajo $i$. Cada operación $O_{ij}$ requiere la máquina $m_{ij}$ durante un tiempo de procesamiento $p_{ij}$ y no puede interrumpirse una vez iniciada. La secuencia de operaciones de cada trabajo representa su ruta tecnológica y debe respetarse estrictamente.

Sea $s_{ij}$ el instante de inicio de la operación $O_{ij}$. Las restricciones de precedencia dentro de cada trabajo se expresan como

$$s_{i,j+1} \geq s_{ij} + p_{ij}, \qquad j = 1, \ldots, m_i - 1.$$

Además, dos operaciones que utilizan la misma máquina no pueden superponerse en el tiempo. Si $O_{ij}$ y $O_{kl}$ requieren la misma máquina, entonces debe cumplirse una de las dos condiciones siguientes:

$$s_{ij} + p_{ij} \leq s_{kl} \qquad \text{o} \qquad s_{kl} + p_{kl} \leq s_{ij}.$$

El instante de término del trabajo $i$ es $C_i = s_{i,m_i} + p_{i,m_i}$, y el makespan se define como

$$C_{\max} = \max_{i=1,\ldots,n} C_i.$$

El objetivo del problema es encontrar una programación factible que minimice $C_{\max}$. En este informe se utilizan tres instancias de complejidad creciente: $6 \times 6$, $15 \times 10$ y $50 \times 10$, donde el primer valor corresponde al número de trabajos y el segundo al número de máquinas.

## 1.2. Metaheurística seleccionada

### 1.2.1. Tipo de metaheurística

El *Simulated Annealing* es una metaheurística de trayectoria, también denominada de solución única. A diferencia de las metaheurísticas poblacionales, como los algoritmos genéticos o la optimización por enjambre de partículas, el *Simulated Annealing* mantiene una sola solución actual y la modifica iterativamente mediante un operador de vecindario (Talbi, 2009). Su funcionamiento se inspira en el proceso físico de recocido de los metales: a altas temperaturas los átomos pueden reorganizarse y aceptar configuraciones de mayor energía; a medida que la temperatura disminuye, el sistema converge hacia configuraciones de menor energía (Kirkpatrick et al., 1983).

En el contexto de optimización, la "energía" corresponde al valor de la función objetivo y la "temperatura" controla la probabilidad de aceptar soluciones peores. Si $x$ es la solución actual, $x'$

es un vecino y $\Delta = f(x') - f(x)$, el criterio de aceptación de Metropolis es

$$P(\text{aceptar } x') = \begin{cases} 1, & \text{si } \Delta < 0, \\ e^{-\Delta/T}, & \text{si } \Delta \geq 0, \end{cases}$$

donde $T > 0$ es la temperatura actual. Cuando $T$ es alta, la probabilidad de aceptar un empeoramiento es considerable y el algoritmo explora ampliamente el espacio de búsqueda. Cuando $T$ se aproxima a cero, la probabilidad de aceptar soluciones peores disminuye y el algoritmo se comporta como una búsqueda local descendente.

### 1.2.2. Ventajas y desventajas

Las principales ventajas del *Simulated Annealing* son:

- Es sencillo de implementar y requiere pocos parámetros.
- Permite escapar de óptimos locales gracias a la aceptación probabilística de soluciones peores.
- Es un algoritmo *anytime*: en cualquier momento de la ejecución se dispone de una solución factible y de la mejor solución encontrada hasta ese instante.
- Se adapta con facilidad a problemas combinatorios en los que es posible definir un vecindario y una función objetivo.
- Tiene un bajo costo computacional por iteración cuando la evaluación del vecindario puede realizarse de forma eficiente.

Sus principales desventajas son:

- No garantiza encontrar la solución óptima ni permite certificar optimalidad.
- Su desempeño es sensible a la calibración de la temperatura inicial, la temperatura mínima y el factor de enfriamiento.
- Un enfriamiento demasiado rápido puede conducir a un óptimo local de baja calidad; un enfriamiento demasiado lento puede consumir un tiempo excesivo.
- Mantiene una sola solución, por lo que no aprovecha la diversidad de una población.
- La calidad de la solución depende del vecindario elegido y de la representación utilizada.

### 1.2.3. Razón de la elección

Se eligió el *Simulated Annealing* por dos razones principales. Primero, el problema estudiado es de naturaleza combinatoria y la metaheurística elegida funciona eficientemente en estos contextos. Segundo, el algoritmo es fácil de explicar, implementar y reproducir, lo que facilita la entrega y la presentación del trabajo.

Además, el *Simulated Annealing* ofrece un buen equilibrio entre calidad y costo computacional para las tres instancias propuestas. Como se muestra en el Capítulo 4, con un presupuesto

de aproximadamente 27.625 iteraciones por ejecución se alcanzó el valor de referencia en las tres instancias, lo que confirma que la elección fue adecuada para el alcance del trabajo.

# 2. IMPLEMENTACIÓN

## 2.1. Arquitectura del software

La implementación se desarrolló en Rust, edición 2024, y se organizó en tres módulos principales: `domain`, `solver` y `ui`. El módulo `domain` contiene las estructuras que representan la instancia y las soluciones: `Jsp`, `Job`, `Operation`, `Schedule` y `ScheduleItem`. El módulo `solver` define los *traits* `Solver` y `SolutionBuilder`, las condiciones de parada y las estrategias de resolución. El módulo `ui` implementa la interfaz de línea de comandos, la barra de progreso y la selección de estrategia.

```
src/
|-- domain/
|   |-- jsp.rs
|   |-- job.rs
|   |-- operation.rs
|   `-- solution/
|       |-- schedule.rs
|       `-- schedule_item.rs
|-- solver/
|   |-- seed.rs
|   |-- solution/
|   |   |-- solution_builder.rs
|   |   |-- solver.rs
|   |   `-- strategies/
|   |       |-- simulated_annealing.rs
|   |       |-- local_search.rs
|   |       |-- random.rs
|   |       |-- swap.rs
|   |       `-- bruteforce.rs
|   `-- stop/
|       |-- cancel.rs
|       |-- timeout.rs
|       `-- stop_handle.rs
`-- ui/
    |-- cli.rs
    |-- progress.rs
    `-- solver_data.rs
```

La interfaz de línea de comandos (CLI) se implementó con la biblioteca `clap`. El programa recibe la ruta de la instancia, un tiempo límite opcional, una semilla opcional y el subcomando

que selecciona la estrategia. Para el *Simulated Annealing*, los parámetros se ingresan mediante opciones con valores por defecto, que además pueden ser especificadas:

```
./target/release/jsp -f data/facil.txt --seed 2026 \
    simulated-annealing
./target/release/jsp -f data/medio.txt --seed 2027 \
    simulated-annealing -i 1000.0 -m 0.001 -a 0.9995
```

La opción global `-seed <U64>` fija la semilla de las estrategias que utilizan azar. Si se omite, el programa genera una semilla a partir de la entropía del sistema. En ambos casos la semilla efectivamente utilizada se imprime en la primera línea de la salida. Las estrategias deterministas, como `bruteforce`, no consumen la semilla, pero el valor elegido se muestra igualmente para dejar registro de la ejecución.

El binario `benchmark` ejecuta automáticamente diez repeticiones por instancia, extrae el makespan desde la salida de la CLI y calcula la media y la desviación estándar muestral. Cada repetición se ejecuta con una semilla explícita e imprime tanto la semilla como el makespan obtenido.

El código fue desarrollado íntegramente por los integrantes del grupo. La única referencia externa es la descripción original del *Simulated Annealing* (Kirkpatrick et al., 1983), citada en el Capítulo 1.

## 2.2. Representación de soluciones

Una solución se representa mediante un vector de identificadores de trabajos, $x = (x_1, \ldots, x_N)$, donde $N = \sum_{i=1}^{n} m_i$ es el número total de operaciones. Cada trabajo $i$ aparece exactamente $m_i$ veces en el vector. Por ejemplo, si una instancia tiene tres trabajos con tres operaciones cada uno, una solución puede ser

$$x = (2, 1, 3, 1, 2, 3, 1, 2, 3).$$

La $k$-ésima aparición del trabajo $i$ en la secuencia indica que debe ejecutarse la $k$-ésima operación de ese trabajo.

## 2.3. Decodificación y evaluación

La función de decodificación recorre la secuencia de izquierda a derecha y mantiene dos arreglos: el instante de término de cada trabajo y el instante de liberación de cada máquina. Para cada identificador de trabajo $i$ se toma su siguiente operación pendiente $O_{ij}$, se calcula

$$s_{ij} = \max\left(\text{término del trabajo } i,\ \text{liberación de la máquina } m_{ij}\right),$$

y se actualizan ambos instantes con $s_{ij} + p_{ij}$. Al finalizar el recorrido, el makespan es el máximo de los instantes de término de los trabajos.

El decodificador se implementó en el *trait* `SolutionBuilder`, con métodos separados para evaluar el makespan y para construir la programación completa de las máquinas. El primero retorna únicamente el valor de la función objetivo; el segundo construye la lista de `ScheduleItem`

con job, operación, máquina, inicio, término y duración, necesaria para mostrar la programación final.

## 2.4. Generación de la solución inicial

La solución inicial se genera mediante una permutación aleatoria del vector base que contiene cada trabajo repetido según su número de operaciones. Para ello se utiliza el generador de números aleatorios de la biblioteca `rand`, inicializado a partir de la semilla de la ejecución. Cada semilla distinta produce una solución inicial distinta, lo que permite medir la variabilidad del algoritmo; una misma semilla reproduce exactamente la misma ejecución.

## 2.5. Vecindario

El operador de vecindario selecciona aleatoriamente dos posiciones $i$ y $j$ de la secuencia, con la condición $x_i \neq x_j$, y las intercambia (Esto evita intercambios que no modifican la solución). Este movimiento se conoce como *swap* y preserva la multiplicidad de cada trabajo, por lo que el vecino siempre es una solución factible. El vecindario completo de una solución está formado por todos los intercambios posibles entre posiciones que contienen trabajos distintos.

## 2.6. Criterio de aceptación

Sea $x$ la solución actual, $x'$ un vecino generado por el operador de swap y $\Delta = f(x') - f(x)$. Si $\Delta < 0$, el vecino mejora la solución y se acepta siempre. Si $\Delta \geq 0$, se acepta con probabilidad $e^{-\Delta/T}$, donde $T$ es la temperatura actual. Este criterio permite que el algoritmo acepte empeoramientos al inicio de la búsqueda y que los rechace con mayor frecuencia a medida que la temperatura disminuye. La mejor solución encontrada se almacena por separado, de modo que un empeoramiento nunca afecta el resultado final.

## 2.7. Esquema de enfriamiento

Se utilizó un enfriamiento geométrico:

$$T_{k+1} = \alpha T_k,$$

con $T_0 = 1000$, $T_{\min} = 0.001$ y $\alpha = 0.9995$. El número de iteraciones hasta alcanzar la temperatura mínima es

$$k_{\max} = \left\lceil \frac{\ln(T_{\min}/T_0)}{\ln \alpha} \right\rceil = \left\lceil \frac{\ln(10^{-6})}{\ln(0.9995)} \right\rceil \approx 27.625.$$

En cada iteración se evalúa un vecino, por lo que una ejecución realiza aproximadamente 27.626 evaluaciones de la función objetivo, incluida la evaluación de la solución inicial. Este presupuesto es fijo e independiente de la instancia, lo que permite comparar las tres instancias bajo las mismas condiciones.

## 2.8. Criterios de parada

El algoritmo se detiene cuando la temperatura alcanza $T_{\min}$. Adicionalmente, la implementación admite dos criterios de parada opcionales: un tiempo límite especificado con la opción `-timeout` y la interrupción manual mediante `Ctrl+C`. Las condiciones de parada se gestionan a través de `StopHandle`, que evalúa todas las condiciones activas y detiene la búsqueda cuando al menos una de ellas se cumple. Cuando esto ocurre, se retorna la mejor solución encontrada hasta ese momento.

## 2.9. Pseudocódigo

El Algoritmo 2.1 resume el funcionamiento del *Simulated Annealing* implementado. La decodificación de una secuencia se denota mediante la función DECODIFICAR, que retorna el makespan correspondiente.

**Algoritmo 2.1** *Simulated Annealing* para el JSP

**Require:** Instancia del JSP, $T_0$, $T_{\min}$, $\alpha$
**Ensure:** Mejor secuencia encontrada $x^*$

Generar una permutación aleatoria $x$ del vector base de trabajos
$f(x) \leftarrow$ DECODIFICAR$(x)$
$x^* \leftarrow x$, $f^* \leftarrow f(x)$
$T \leftarrow T_0$
**while** $T > T_{\min}$ y no se cumpla un criterio de parada **do**
    Elegir dos posiciones aleatorias $i, j$ con $x_i \neq x_j$
    $x' \leftarrow x$ con las posiciones $i$ y $j$ intercambiadas
    $f(x') \leftarrow$ DECODIFICAR$(x')$
    $\Delta \leftarrow f(x') - f(x)$
    **if** $\Delta < 0$ **o** aleatorio$(0, 1) < e^{-\Delta/T}$ **then**
        $x \leftarrow x'$
        **if** $f(x) < f^*$ **then**
            $x^* \leftarrow x$
            $f^* \leftarrow f(x)$
        **end if**
    **end if**
    $T \leftarrow \alpha T$
**end while**
**return** $x^*$

## 2.10. Diagrama de flujo

La Figura 2.1 resume el flujo del *Simulated Annealing* implementado, desde la generación de la solución inicial hasta la devolución de la mejor secuencia encontrada.

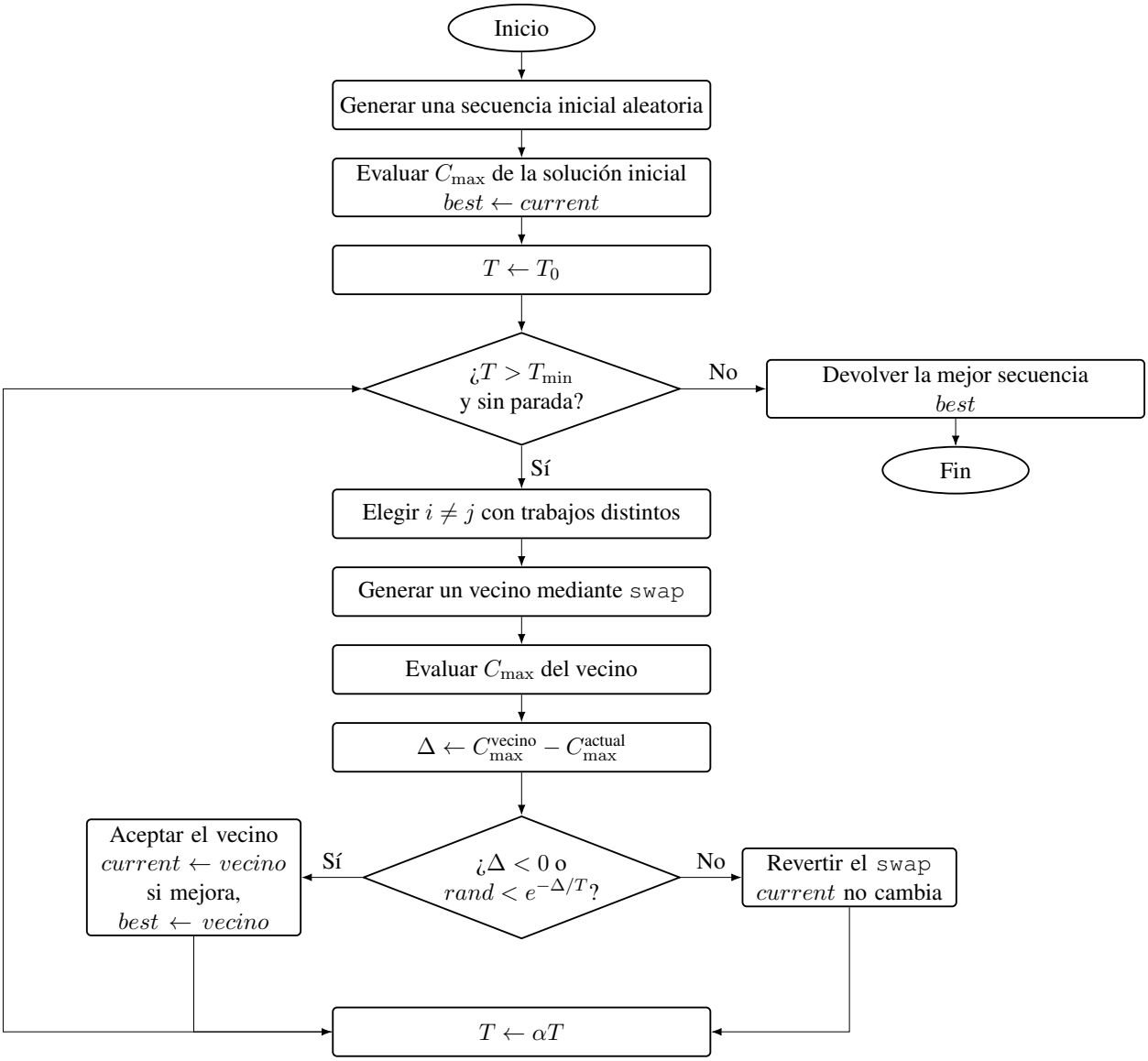



Figura 2.1: Diagrama de flujo del *Simulated Annealing* implementado.

## 2.11. Parámetros y presupuesto computacional

La Tabla 2.1 resume los parámetros utilizados en todos los experimentos. No se realizó ajuste específico por instancia: las tres instancias se resolvieron con el mismo conjunto de parámetros.

El presupuesto total de la experimentación fue de $3 \times 10 \times 27.626 = 828.780$ evaluaciones de la función objetivo. Los tiempos de ejecución medidos en el equipo de pruebas fueron aproximadamente 0,007 s para la instancia fácil, 0,011 s para la instancia media y 0,028 s para la instancia difícil, incluyendo el tiempo de arranque del proceso. Estos valores corresponden a un procesador Intel Core i3-1215U con 8 GB de memoria RAM, ejecutados sobre Arch Linux con kernel 7.2.14-arch1-1 y arquitectura x86_64, utilizando Rust 1.99.1 (Cargo 1.99.1).

| Parámetro | Símbolo | Valor |
|---|---|---|
| Temperatura inicial | $T_0$ | 1000 |
| Temperatura mínima | $T_{\min}$ | 0,001 |
| Factor de enfriamiento | $\alpha$ | 0,9995 |
| Iteraciones por ejecución | $k_{\max}$ | 27.625 |
| Evaluaciones por ejecución | – | 27.626 |
| Ejecuciones por instancia | – | 10 |
| Tiempo límite | – | Desactivado |
| Semilla | – | 2026–2035 (`benchmark`) |
| Vecindario | – | Swap de dos posiciones |
| Criterio de aceptación | – | Metropolis |

Tabla 2.1: Parámetros del *Simulated Annealing* utilizado en los experimentos.

## 2.12. Reproducibilidad

El repositorio incluye un archivo `README.txt` con las instrucciones de compilación y ejecución, además de la identificación de los integrantes del grupo. Para compilar y ejecutar el algoritmo se utilizan los siguientes comandos:

```
cargo build --release

./target/release/jsp -f data/facil.txt --seed 2026 \
    simulated-annealing
./target/release/jsp -f data/medio.txt --seed 2026 \
    simulated-annealing
./target/release/jsp -f data/dificil.txt --seed 2026 \
    simulated-annealing
```

Para reproducir la tabla completa de diez ejecuciones por instancia se ejecuta:

```
./target/release/benchmark
```

El binario `benchmark` recorre las instancias `facil.txt`, `medio.txt` y `dificil.txt`, y ejecuta el *Simulated Annealing* diez veces por instancia con las semillas 2026 a 2035. Para cada repetición imprime la semilla utilizada y el makespan obtenido, y al final reporta el mejor valor, el peor valor, la media y la desviación estándar. Esto permite reproducir cada resultado de forma individual ejecutando la CLI con la semilla correspondiente.

# 3. MEJOR SOLUCIÓN

Para cada instancia se registró la mejor solución encontrada entre las diez ejecuciones independientes. La Tabla 3.1 presenta el makespan de la mejor solución, el valor de referencia y la brecha correspondiente. Los valores de referencia se usan únicamente para calcular la brecha y no forman parte de la metaheurística implementada. En las instancias media y difícil, el mejor makespan coincide con la cota inferior dada por la máquina más cargada, por lo que dichas soluciones son óptimas.

| Instancia | Tamaño | Mejor $C_{\max}$ | Referencia | Brecha | Ejecuciones en la referencia |
|---|---|---|---|---|---|
| Fácil | $6 \times 6$ | 55 | 55 | 0,00 % | 10/10 |
| Medio | $15 \times 10$ | 1032 | 1032 | 0,00 % | 5/10 |
| Difícil | $50 \times 10$ | 2794 | 2794 | 0,00 % | 10/10 |

Tabla 3.1: Mejor solución encontrada para cada instancia.

La mejor solución de la instancia fácil se obtuvo en las diez ejecuciones, con un makespan de 55. En la instancia media, el valor de referencia de 1032 se alcanzó en cinco de las diez ejecuciones; las cinco restantes quedaron entre 1036 y 1069. En la instancia difícil, las diez ejecuciones alcanzaron el valor 2794.

La brecha se calculó como

$$\text{brecha} = \frac{C_{\max}^{\text{mejor}} - C_{\max}^{\text{ref}}}{C_{\max}^{\text{ref}}} \times 100\,\%.$$

En las tres instancias la mejor solución encontrada coincide con el valor de referencia, por lo que la brecha es cero. En las instancias media y difícil, además, ese valor coincide con la cota inferior dada por la máquina más cargada, lo que prueba que la solución encontrada es óptima. La diferencia entre ejecuciones se analiza con mayor detalle en el Capítulo 4.

# 4. RESULTADOS

## 4.1. Configuración experimental

Los experimentos se realizaron con el binario compilado en modo `release` (optimización máxima del binario). Para cada una de las tres instancias se ejecutó el *Simulated Annealing* diez veces de forma independiente, utilizando los parámetros de la Tabla 2.1 y sin tiempo límite. Cada ejecución parte de una permutación inicial aleatoria distinta, por lo que los resultados reflejan la variabilidad propia del algoritmo.

El equipo de pruebas fue un computador con procesador Intel Core i3-1215U y 8 GB de memoria RAM, con sistema operativo Arch Linux, kernel 7.2.14-arch1-1 y arquitectura x86_64. La implementación utilizó Rust 1.99.1 (Cargo 1.99.1) y las bibliotecas `clap`, `rand`, `indicatif`, `ctrlc` y `thiserror`. El binario `benchmark` automatizó las treinta ejecuciones y el cálculo de los estadísticos.

## 4.2. Resultados por ejecución

La Tabla 4.1 presenta el makespan obtenido en cada una de las diez ejecuciones para las tres instancias, junto con la media y la desviación estándar muestral.

| Ejecución | Fácil ($6 \times 6$) | Medio ($15 \times 10$) | Difícil ($50 \times 10$) |
|---|---|---|---|
| 1 | 55 | 1069 | 2794 |
| 2 | 55 | 1032 | 2794 |
| 3 | 55 | 1037 | 2794 |
| 4 | 55 | 1032 | 2794 |
| 5 | 55 | 1032 | 2794 |
| 6 | 55 | 1032 | 2794 |
| 7 | 55 | 1036 | 2794 |
| 8 | 55 | 1036 | 2794 |
| 9 | 55 | 1032 | 2794 |
| 10 | 55 | 1051 | 2794 |
| Media | 55,00 | 1038,90 | 2794,00 |
| Desviación estándar | 0,00 | 12,07 | 0,00 |

Tabla 4.1: Makespan obtenido en diez ejecuciones independientes por instancia.

## 4.3. Resumen estadístico

La Tabla 4.2 resume los principales estadísticos por instancia y los compara con el valor de referencia. La brecha media se calcula como la diferencia porcentual entre la media de las diez ejecuciones y el valor de referencia.

| Instancia | Referencia | Mejor | Peor | Media | Desv. est. | Brecha media |
|---|---|---|---|---|---|---|
| Fácil | 55 | 55 | 55 | 55,00 | 0,00 | 0,00 % |
| Medio | 1032 | 1032 | 1069 | 1038,90 | 12,07 | 0,67 % |
| Difícil | 2794 | 2794 | 2794 | 2794,00 | 0,00 | 0,00 % |

Tabla 4.2: Resumen estadístico de las diez ejecuciones por instancia.

## 4.4. Análisis de resultados

En la instancia fácil, las diez ejecuciones alcanzaron el mismo makespan de 55, que coincide con el valor de referencia. La desviación estándar nula indica que el algoritmo fue completamente estable en esta instancia. El tamaño reducido del problema ($6 \times 6$, 36 operaciones) permite que el presupuesto de 27.625 iteraciones explore el espacio de búsqueda con suficiente profundidad, incluso partiendo de soluciones iniciales aleatorias distintas.

En la instancia media, cinco de las diez ejecuciones alcanzaron el valor de referencia de 1032, mientras que las cinco restantes obtuvieron valores entre 1036 y 1069. La media fue 1038,90 y la desviación estándar 12,07, lo que representa una brecha media de solo 0,67 % respecto del valor de referencia. La mayor variabilidad se explica por el tamaño del problema ($15 \times 10$, 150 operaciones) y por la naturaleza estocástica del *Simulated Annealing*: la calidad del óptimo local alcanzado depende de la trayectoria seguida y de la solución inicial. Aun así, el peor resultado (1069) se encuentra a solo 3,59 % del valor de referencia, lo que muestra que el algoritmo mantiene una calidad aceptable incluso en sus ejecuciones menos favorables.

En la instancia difícil, las diez ejecuciones alcanzaron el valor 2794, que coincide con la cota inferior determinada por la máquina más cargada. La desviación estándar nula y la coincidencia con el valor de referencia indican que, para esta instancia, la estructura del problema guía al algoritmo hacia una solución de alta calidad de forma consistente. El mayor número de operaciones ($50 \times 10$, 500 operaciones) no impidió alcanzar el valor de referencia, aunque el costo por evaluación es mayor que en las otras instancias.

En términos generales, el *Simulated Annealing* mostró un comportamiento robusto: con un único conjunto de parámetros y sin ajuste por instancia, coincidió con el valor de referencia en las tres instancias y mantuvo una brecha media inferior al 1 % en el caso más variable. El presupuesto de aproximadamente 27.625 iteraciones por ejecución resultó suficiente para las instancias estudiadas y el costo computacional fue bajo: menos de 0,03 s por ejecución en el equipo de pruebas.

## 4.5. Gráfico de cajas

La Figura 4.1 muestra la distribución de los makespan obtenidos en las diez ejecuciones de cada instancia. Cada panel utiliza una escala independiente para evitar que las diferencias de magnitud entre instancias oculten la dispersión. Los puntos negros corresponden a cada ejecución individual y la caja resume los cuartiles y la mediana.

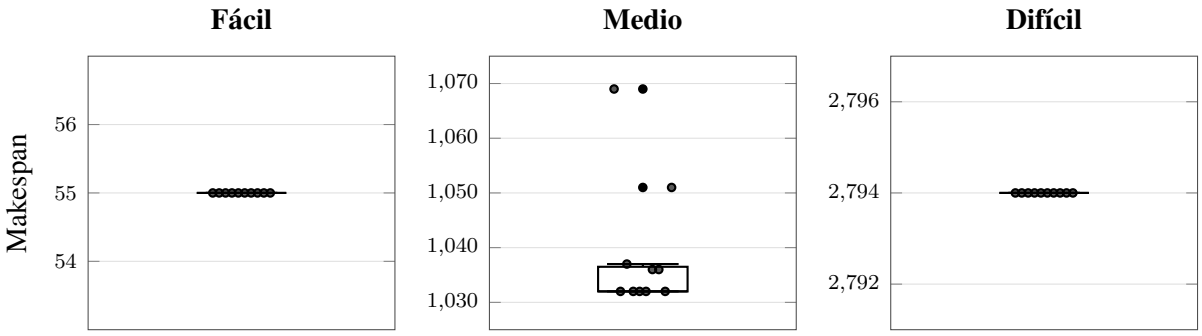



Figura 4.1: Gráfico de cajas del makespan obtenido en diez ejecuciones por instancia.

La instancia fácil y la instancia difícil presentan una dispersión nula, por lo que sus cajas se reducen a una línea en el valor de referencia. La instancia media concentra la mayor dispersión, con cinco ejecuciones en 1032 y cinco valores superiores.

# 5. CONCLUSIONES

En este trabajo se implementó y evaluó la metaheurística de *Simulated Annealing* para resolver Job Shop Problem. El algoritmo se desarrolló íntegramente en Rust, con una representación de soluciones basada en secuencias de trabajos y un decodificador que garantiza el cumplimiento de las precedencias y de la capacidad de las máquinas. Se evaluaron tres instancias de complejidad creciente y se realizaron diez ejecuciones independientes por instancia.

Los resultados muestran que la mejor solución encontrada coincidió con el valor de referencia en las tres instancias: 55 para la instancia fácil, 1032 para la instancia media y 2794 para la instancia difícil. La instancia media presentó la mayor variabilidad, con una media de 1038,90 y una desviación estándar de 12,07, mientras que las instancias fácil y difícil alcanzaron ese valor en las diez ejecuciones. La brecha media en el caso más variable fue inferior al 1 %, lo que confirma la calidad del algoritmo con un presupuesto computacional reducido.

## 5.1. ¿Qué aprendió implementando la metaheurística?

La implementación permitió comprender que el desempeño de una metaheurística no depende únicamente del algoritmo, sino también de la representación de las soluciones, de la función de decodificación y del vecindario. Una representación adecuada puede garantizar la factibilidad de todas las soluciones y simplificar enormemente la búsqueda. También se aprendió que el esquema de enfriamiento y el criterio de aceptación son fundamentales para equilibrar exploración y explotación, y que la experimentación sistemática con varias ejecuciones es necesaria para medir la variabilidad del método.

## 5.2. ¿Cuál fue el desafío más grande de implementar la metaheurística?

El desafío más grande fue lograr comprender la metaheurística seleccionada al nivel de implementarla en código. Posteriormente, la calibración de los parámetros fue otro desafio. Un enfriamiento demasiado rápido reduce la exploración y puede estancar el algoritmo en un óptimo local; un enfriamiento demasiado lento incrementa innecesariamente el tiempo de ejecución. Encontrar un equilibrio adecuado requirió probar distintas configuraciones y analizar su efecto sobre la calidad de las soluciones.

## 5.3. ¿Tiene alguna idea para mejorar los resultados?

Existen varias líneas de mejora. En primer lugar, se podría utilizar un vecindario más informado, por ejemplo, intercambios basados en la ruta crítica o movimientos de inserción en lugar de swaps aleatorios. En segundo lugar, se podría implementar un esquema de enfriamiento adaptativo o con recalentamientos periódicos para escapar de óptimos locales profundos. En tercer lugar, se podría combinar el *Simulated Annealing* con una búsqueda local intensiva al final de la ejecución, o ejecutar varias réplicas en paralelo y quedarse con la mejor solución. Finalmente, se podrían ajustar los parámetros por instancia mediante un estudio experimental más amplio o utilizar un

enfoque de *tuning* automático.

## 5.4. ¿Cree que este tipo de técnicas puede ser aplicada en la realidad chilena?

Sí. Las metaheurísticas como el *Simulated Annealing* son especialmente útiles en contextos donde los métodos exactos no son viables por el tamaño o la complejidad de los problemas. En Chile, este tipo de técnicas puede aplicarse en la planificación de la producción de pequeñas y medianas empresas manufactureras, en la programación de talleres de mantenimiento, en la gestión de turnos y en la logística de distribución. La principal barrera no es el algoritmo, sino la disponibilidad de datos confiables y la capacidad de integrar la solución con los sistemas de información existentes.

Además, el bajo costo computacional del *Simulated Annealing* permite ejecutarlo en computadores personales o en servidores modestos, sin necesidad de infraestructura especializada. Esto lo convierte en una alternativa accesible para organizaciones que no pueden invertir en soluciones comerciales de optimización. Con una adecuada capacitación y un proceso de validación con datos reales, este tipo de técnicas podría aportar mejoras concretas en eficiencia y en tiempos de respuesta.

## 5.5. Conclusión general

El *Simulated Annealing* demostró ser una metaheurística adecuada para el problema estudiado: simple de implementar, fácil de parametrizar y capaz de alcanzar el valor de referencia en las tres instancias con un presupuesto computacional reducido. La combinación de una representación factible, un decodificador eficiente y un criterio de aceptación probabilístico permitió obtener resultados de alta calidad y una variabilidad controlada. El trabajo deja abiertas varias líneas de mejora que podrían aumentar la robustez del algoritmo en instancias de mayor tamaño o en problemas con restricciones adicionales.

# REFERENCIAS

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, *220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Talbi, E.-G. (2009). *Metaheuristics: From design to implementation*. John Wiley & Sons.